

[INICIAL] TCArg 2026 Contest #4

A. Array Eversion

1 second, 256 megabytes

You are given an array a of length n .

Let's define the *eversion* operation. Let $x = a_n$. Then array a is partitioned into two parts: left and right. The left part contains the elements of a that are not greater than x ($\leq x$). The right part contains the elements of a that are strictly greater than x ($> x$). The order of elements in each part is kept the same as before the operation, i. e. the partition is stable. Then the array is replaced with the concatenation of the left and the right parts.

For example, if the array a is $[2, 4, 1, 5, 3]$, the eversion goes like this: $[2, 4, 1, 5, 3] \rightarrow [2, 1, 3], [4, 5] \rightarrow [2, 1, 3, 4, 5]$.

We start with the array a and perform eversions on this array. We can prove that after several eversions the array a stops changing. Output the minimum number k such that the array stops changing after k eversions.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). Description of the test cases follows.

The first line contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$).

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case print a single integer k — the number of eversions after which the array stops changing.

input
3
5
2 4 1 5 3
5
5 3 2 4 1
4
1 1 1 1
output
1
2
0

Consider the fist example.

- The first eversion: $a = [1, 4, 2, 5, 3]$, $x = 3$. $[2, 4, 1, 5, 3] \rightarrow [2, 1, 3], [4, 5] \rightarrow [2, 1, 3, 4, 5]$.
- The second and following eversions: $a = [2, 1, 3, 4, 5]$, $x = 5$. $[2, 1, 3, 4, 5] \rightarrow [2, 1, 3, 4, 5], [] \rightarrow [2, 1, 3, 4, 5]$. This eversion does not change the array, so the answer is 1.

Consider the second example.

- The first eversion: $a = [5, 3, 2, 4, 1]$, $x = 1$. $[5, 3, 2, 4, 1] \rightarrow [1], [5, 3, 2, 4] \rightarrow [1, 5, 3, 2, 4]$.
- The second eversion: $a = [1, 5, 3, 2, 4]$, $x = 4$. $[1, 5, 3, 2, 4] \rightarrow [1, 3, 2, 4], [5] \rightarrow [1, 3, 2, 4, 5]$.
- The third and following eversions: $a = [1, 3, 2, 4, 5]$, $x = 5$. $[1, 3, 2, 4, 5] \rightarrow [1, 3, 2, 4, 5], [] \rightarrow [1, 3, 2, 4, 5]$. This eversion does not change the array, so the answer is 2.

B. Inscribed Figures

1 second, 256 megabytes

The math faculty of Berland State University has suffered the sudden drop in the math skills of enrolling students. This year the highest grade on the entrance math test was 8. Out of 100! Thus, the decision was made to make the test easier.

Future students will be asked just a single question. They are given a sequence of integer numbers $a_1, a_2, \dots, a_n$, each number is from 1 to 3 and $a_i \neq a_{i+1}$ for each valid i . The i -th number represents a type of the i -th figure:

1. circle;
2. isosceles triangle with the length of height equal to the length of base;
3. square.

The figures of the given sequence are placed somewhere on a Cartesian plane in such a way that:

- $(i + 1)$ -th figure is inscribed into the i -th one;
- each triangle base is parallel to OX;
- the triangle is oriented in such a way that the vertex opposite to its base is at the top;
- each square sides are parallel to the axes;

- for each i from 2 to n figure i has the maximum possible length of side for triangle and square and maximum radius for circle.

Note that the construction is unique for some fixed position and size of just the first figure.

The task is to calculate the number of **distinct** points (not necessarily with integer coordinates) where figures touch. The trick is, however, that the number is sometimes infinite. But that won't make the task difficult for you, will it?

So can you pass the math test and enroll into Berland State University?

Input

The first line contains a single integer n ($2 \leq n \leq 100$) — the number of figures.

The second line contains n integer numbers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 3$, $a_i \neq a_{i+1}$) — types of the figures.

Output

The first line should contain either the word "Infinite" if the number of distinct points where figures touch is infinite or "Finite" otherwise.

If the number is finite than print it in the second line. It's guaranteed that the number fits into 32-bit integer type.

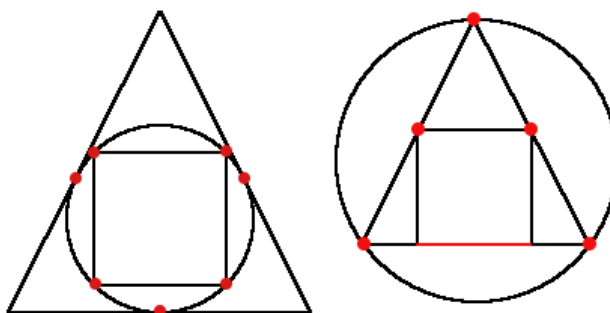
input
3 2 1 3
output
Finite 7

input
3 1 2 3
output
Infinite

Here are the glorious pictures for the examples. Note that the triangle is not equilateral but just isosceles with the length of height equal to the length of base. Thus it fits into a square in a unique way.

The distinct points where figures touch are marked red.

In the second example the triangle and the square touch each other for the whole segment, it contains infinite number of points.



C. A and B and Lecture Rooms

2 seconds, 256 megabytes

A and B are preparing themselves for programming contests.

The University where A and B study is a set of rooms connected by corridors. Overall, the University has n rooms connected by $n - 1$ corridors so that you can get from any room to any other one by moving along the corridors. The rooms are numbered from 1 to n .

Every day A and B write contests in some rooms of their university, and after each contest they gather together in the same room and discuss problems. A and B want the distance from the rooms where problems are discussed to the rooms where contests are written to be equal. The distance between two rooms is the number of edges on the shortest path between them.

As they write contests in new rooms every day, they asked you to help them find the number of possible rooms to discuss problems for each of the following m days.

Input

The first line contains integer n ($1 \leq n \leq 10^5$) — the number of rooms in the University.

The next $n - 1$ lines describe the corridors. The i -th of these lines ($1 \leq i \leq n - 1$) contains two integers a_i and b_i ($1 \leq a_i, b_i \leq n$), showing that the i -th corridor connects rooms a_i and b_i .

The next line contains integer m ($1 \leq m \leq 10^5$) — the number of queries.

Next m lines describe the queries. The j -th of these lines ($1 \leq j \leq m$) contains two integers x_j and y_j ($1 \leq x_j, y_j \leq n$) that means that on the j -th day A will write the contest in the room x_j , B will write in the room y_j .

Output

In the i -th ($1 \leq i \leq m$) line print the number of rooms that are equidistant from the rooms where A and B write contest on the i -th day.

input
4 1 2 1 3 2 4 1 2 3
output
1

input
4 1 2 2 3 2 4 2 1 2 1 3
output
0 2

in the first sample there is only one room at the same distance from rooms number 2 and 3 — room number 1.

D. Berry Jam

2 seconds, 256 megabytes

Karlsson has recently discovered a huge stock of berry jam jars in the basement of the house. More specifically, there were $2n$ jars of strawberry and blueberry jam.

All the $2n$ jars are arranged in a row. The stairs to the basement are exactly in the middle of that row. So when Karlsson enters the basement, he sees exactly n jars to his left and n jars to his right.

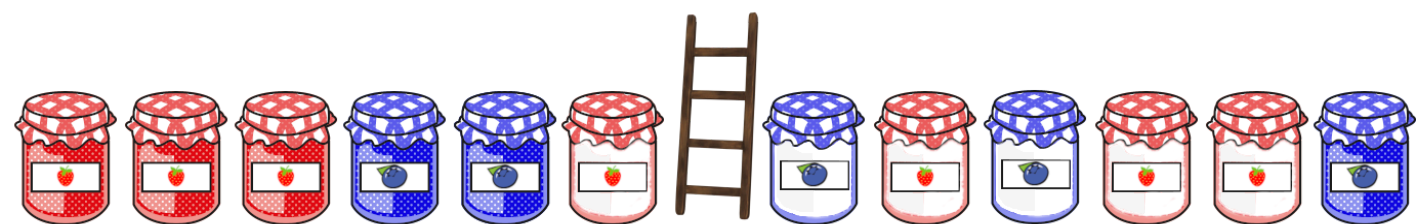
For example, the basement might look like this:



Being the starightforward man he is, he immediately starts eating the jam. In one minute he chooses to empty either the first non-empty jar to his left or the first non-empty jar to his right.

Finally, Karlsson decided that at the end the amount of full strawberry and blueberry jam jars should become the same.

For example, this might be the result:



He has eaten 1 jar to his left and then 5 jars to his right. There remained exactly 3 full jars of both strawberry and blueberry jam. Jars are numbered from 1 to $2n$ from left to right, so Karlsson initially stands between jars n and $n + 1$.

What is the minimum number of jars Karlsson is required to empty so that an equal number of full strawberry and blueberry jam jars is left?

Your program should answer t independent test cases.

Input

The first line contains one integer t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 10^5$).

The second line of each test case contains $2n$ integers $a_1, a_2, \dots, a_{2n}$ ($1 \leq a_i \leq 2$) — $a_i = 1$ means that the i -th jar from the left is a strawberry jam jar and $a_i = 2$ means that it is a blueberry jam jar.

It is guaranteed that the sum of n over all test cases does not exceed 10^5 .

Output

For each test case print the answer to it — the minimum number of jars Karlsson is required to empty so that an equal number of full strawberry and blueberry jam jars is left.

input
4 6 1 1 1 2 2 1 2 1 2 1 1 2 2 1 2 1 2 3 1 1 1 1 1 1 2 2 1 1 1
output
6 0 6 2

The picture from the statement describes the first test case.

In the second test case the number of strawberry and blueberry jam jars is already equal.

In the third test case Karlsson is required to eat all 6 jars so that there remain 0 jars of both jams.

In the fourth test case Karlsson can empty either the second and the third jars or the third and the fourth one. The both scenarios will leave 1 jar of both jams.

E. Minimal Diameter Forest

2 seconds, 256 megabytes

You are given a forest — an undirected graph with n vertices such that each its connected component is a tree.

The diameter (aka "longest shortest path") of a connected undirected graph is the maximum number of edges in the **shortest** path between any pair of its vertices.

You task is to add some edges (possibly zero) to the graph so that it becomes a tree and the diameter of the tree is minimal possible.

If there are multiple correct answers, print any of them.

Input

The first line contains two integers n and m ($1 \leq n \leq 1000, 0 \leq m \leq n - 1$) — the number of vertices of the graph and the number of edges, respectively.

Each of the next m lines contains two integers v and u ($1 \leq v, u \leq n, v \neq u$) — the descriptions of the edges.

It is guaranteed that the given graph is a forest.

Output

In the first line print the diameter of the resulting tree.

Each of the next $(n - 1) - m$ lines should contain two integers v and u ($1 \leq v, u \leq n, v \neq u$) — the descriptions of the **added edges**.

The resulting graph should be a tree and its diameter should be minimal possible.

For $m = n - 1$ no edges are added, thus the output consists of a single integer — diameter of the given tree.

If there are multiple correct answers, print any of them.

input
4 2 1 2 2 3
output
2 4 2

input
2 0

output
1 1 2

input
3 2 1 3 2 3
output
2

In the first example adding edges (1, 4) or (3, 4) will lead to a total diameter of 3. Adding edge (2, 4), however, will make it 2.

Edge (1, 2) is the only option you have for the second example. The diameter is 1.

You can't add any edges in the third example. The diameter is already 2.

F. Sweet Problem

1 second, 256 megabytes

You have three piles of candies: red, green and blue candies:

- the first pile contains only red candies and there are r candies in it,
- the second pile contains only green candies and there are g candies in it,
- the third pile contains only blue candies and there are b candies in it.

Each day Tanya eats exactly two candies of different colors. She is free to choose the colors of eaten candies: the only restriction that she can't eat two candies of the same color in a day.

Find the maximal number of days Tanya can eat candies? Each day she needs to eat **exactly** two candies.

Input

The first line contains integer t ($1 \leq t \leq 1000$) — the number of test cases in the input. Then t test cases follow.

Each test case is given as a separate line of the input. It contains three integers r, g and b ($1 \leq r, g, b \leq 10^8$) — the number of red, green and blue candies, respectively.

Output

Print t integers: the i -th printed integer is the answer on the i -th test case in the input.

input
6 1 1 1 1 2 1 4 1 1 7 4 10 8 1 4 8 2 8
output
1 2 2 10 5 9

In the first example, Tanya can eat candies for one day only. She can eat any pair of candies this day because all of them have different colors.

In the second example, Tanya can eat candies for two days. For example, she can eat red and green candies on the first day, and green and blue candies on the second day.

In the third example, Tanya can eat candies for two days. For example, she can eat red and green candies on the first day, and red and blue candies on the second day. Note, that two red candies will remain uneaten.

G. Life of a Flower

1 second, 256 megabytes

Petya has got an interesting flower. Petya is a busy person, so he sometimes forgets to water it. You are given n days from Petya's live and you have to determine what happened with his flower in the end.

The flower grows as follows:

- If the flower isn't watered for two days in a row, it dies.
- If the flower is watered in the i -th day, it grows by 1 centimeter.

- If the flower is watered in the i -th and in the $(i - 1)$ -th day ($i > 1$), then it grows by 5 centimeters instead of 1.
- If the flower is not watered in the i -th day, it does not grow.

At the beginning of the 1-st day the flower is 1 centimeter tall. What is its height after n days?

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). Description of the test cases follows.

The first line of each test case contains the only integer n ($1 \leq n \leq 100$).

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($a_i = 0$ or $a_i = 1$). If $a_i = 1$, the flower is watered in the i -th day, otherwise it is not watered.

Output

For each test case print a single integer k — the flower's height after n days, or -1 , if the flower dies.

input
4 3 1 0 1 3 0 1 1 4 1 0 0 1 1 0
output
3 7 -1 1

H. 24 Game

1 second, 256 megabytes

Little X used to play a card game called "24 Game", but recently he has found it too easy. So he invented a new game.

Initially you have a sequence of n integers: 1, 2, ..., n . In a single step, you can pick two of them, let's denote them a and b , erase them from the sequence, and append to the sequence either $a + b$, or $a - b$, or $a \times b$.

After $n - 1$ steps there is only one number left. Can you make this number equal to 24?

Input

The first line contains a single integer n ($1 \leq n \leq 10^5$).

Output

If it's possible, print "YES" in the first line. Otherwise, print "NO" (without the quotes).

If there is a way to obtain 24 as the result number, in the following $n - 1$ lines print the required operations an operation per line. Each operation should be in form: " $a \text{ op } b = c$ ". Where a and b are the numbers you've picked at this operation; op is either "+", or "-", or "*"; c is the result of corresponding operation. Note, that the absolute value of c mustn't be greater than 10^{18} . The result of the last operation must be equal to 24. Separate operator sign and equality sign from numbers with spaces.

If there are multiple valid answers, you may print any of them.

input
1
output
NO

input
8
output
YES 8 * 7 = 56 6 * 5 = 30 3 - 4 = -1 1 - 2 = -1 30 - -1 = 31 56 - 31 = 25 25 + -1 = 24

I. Coder

Iahub likes chess very much. He even invented a new chess piece named **Coder**. A **Coder** can move (and attack) one square horizontally or vertically. More precisely, if the **Coder** is located at position (x, y) , he can move to (or attack) positions $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$ and $(x, y - 1)$.

Iahub wants to know the maximum number of **Coders** that can be placed on an $n \times n$ chessboard, so that no **Coder** attacks any other **Coder**.

Input

The first line contains an integer n ($1 \leq n \leq 1000$).

Output

On the first line print an integer, the maximum number of **Coders** that can be placed on the chessboard.

On each of the next n lines print n characters, describing the configuration of the **Coders**. For an empty cell print an '.', and for a **Coder** print a 'C'.

If there are multiple correct answers, you can print any.

input
2
output
2 C. .C

J. Closing The Gap

2 seconds, 256 megabytes

There are n block towers in a row, where tower i has a height of a_i . You're part of a building crew, and you want to make the buildings look as nice as possible. In a single day, you can perform the following operation:

- Choose two indices i and j ($1 \leq i, j \leq n$; $i \neq j$), and move a block from tower i to tower j . This essentially decreases a_i by 1 and increases a_j by 1.

You think the ugliness of the buildings is the height difference between the tallest and shortest buildings. Formally, the ugliness is defined as $\max(a) - \min(a)$.

What's the minimum possible ugliness you can achieve, after any number of days?

Input

The first line contains one integer t ($1 \leq t \leq 1000$) — the number of test cases. Then t cases follow.

The first line of each test case contains one integer n ($2 \leq n \leq 100$) — the number of buildings.

The second line of each test case contains n space separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^7$) — the heights of the buildings.

Output

For each test case, output a single integer — the minimum possible ugliness of the buildings.

input
3 3 10 10 10 4 3 2 1 2 5 1 2 3 1 5
output
0 0 1

In the first test case, the ugliness is already 0.

In the second test case, you should do one operation, with $i = 1$ and $j = 3$. The new heights will now be $[2, 2, 2, 2]$, with an ugliness of 0.

In the third test case, you may do three operations:

- with $i = 3$ and $j = 1$. The new array will now be $[2, 2, 2, 1, 5]$,
- with $i = 5$ and $j = 4$. The new array will now be $[2, 2, 2, 2, 4]$,
- with $i = 5$ and $j = 3$. The new array will now be $[2, 2, 3, 2, 3]$.

The resulting ugliness is 1. It can be proven that this is the minimum possible ugliness for this test.

K. Train Maintenance

1 second, 512 megabytes

Kawasiro Nitōri is excellent in engineering. Thus she has been appointed to help maintain trains.

There are n models of trains, and Nitōri's department will only have at most one train of each model at any moment. In the beginning, there are no trains, at each of the following m days, one train will be added, or one train will be removed. When a train of model i is added at day t , it works for x_i days (day t inclusive), then it is in maintenance for y_i days, then in work for x_i days again, and so on until it is removed.

In order to make management easier, Nitōri wants you to help her calculate how many trains are in maintenance in each day.

On a day a train is removed, it is not counted as in maintenance.

Input

The first line contains two integers n, m ($1 \leq n, m \leq 2 \cdot 10^5$).

The i -th of the next n lines contains two integers x_i, y_i ($1 \leq x_i, y_i \leq 10^9$).

Each of the next m lines contains two integers op, k ($1 \leq k \leq n, op = 1$ or $op = 2$). If $op = 1$, it means this day's a train of model k is added, otherwise the train of model k is removed. It is guaranteed that when a train of model x is added, there is no train of the same model in the department, and when a train of model x is removed, there is such a train in the department.

Output

Print m lines, The i -th of these lines contains one integers, denoting the number of trains in maintenance in the i -th day.

input
<pre> 3 4 10 15 12 10 1 1 1 3 1 1 2 1 2 3 </pre>
output
<pre> 0 1 0 0 </pre>

input
<pre> 5 4 1 1 100000000 100000000 998244353 1 2 1 1 2 1 5 2 5 1 5 1 1 </pre>
output
<pre> 0 0 0 1 </pre>

Consider the first example:

The first day: Nitōri adds a train of model 3. Only a train of model 3 is running and no train is in maintenance.

The second day: Nitōri adds a train of model 1. A train of model 1 is running and a train of model 3 is in maintenance.

The third day: Nitōri removes a train of model 1. The situation is the same as the first day.

The fourth day: Nitōri removes a train of model 3. There are no trains at all.

L. Berland National Library

1 second, 256 megabytes

Berland National Library has recently been built in the capital of Berland. In addition, in the library you can take any of the collected works of Berland leaders, the library has a *reading room*.

Today was the pilot launch of an automated reading room visitors' accounting system! The scanner of the system is installed at the entrance to the reading room. It records the events of the form "reader entered room", "reader left room". Every reader is assigned a *registration number* during the registration procedure at the library — it's a unique integer from 1 to 10^6 . Thus, the system logs events of two forms:

- "+ r_i " — the reader with registration number r_i entered the room;
- "- r_i " — the reader with registration number r_i left the room.

The first launch of the system was a success, it functioned for some period of time, and, at the time of its launch and at the time of its shutdown, the reading room may already have visitors.

Significant funds of the budget of Berland have been spent on the design and installation of the system. Therefore, some of the citizens of the capital now demand to explain the need for this system and the benefits that its implementation will bring. Now, the developers of the system need to urgently come up with reasons for its existence.

Help the system developers to find the minimum possible capacity of the reading room (in visitors) using the log of the system available to you.

Input

The first line contains a positive integer n ($1 \leq n \leq 100$) — the number of records in the system log. Next follow n events from the system journal in the order in which they were made. Each event was written on a single line and looks as "+ r_i " or "- r_i ", where r_i is an integer from 1 to 10^6 , the registration number of the visitor (that is, distinct visitors always have distinct registration numbers).

It is guaranteed that the log is not contradictory, that is, for every visitor the types of any of his two consecutive events are distinct. Before starting the system, and after stopping the room may possibly contain visitors.

Output

Print a single integer — the minimum possible capacity of the reading room.

input
6 + 12001 - 12001 - 1 - 1200 + 1 + 7
output
3

input
2 - 1 - 2
output
2

input
2 + 1 - 1
output
1

In the first sample test, the system log will ensure that at some point in the reading room were visitors with registration numbers 1, 1200 and 12001. More people were not in the room at the same time based on the log. Therefore, the answer to the test is 3.

M. Mysterious Code

2 seconds, 256 megabytes

During a normal walk in the forest, Katie has stumbled upon a mysterious code! However, the mysterious code had some characters unreadable. She has written down this code as a string c consisting of lowercase English characters and asterisks ("*"), where each of the asterisks denotes an unreadable character. Excited with her discovery, Katie has decided to recover the unreadable characters by replacing each asterisk with arbitrary lowercase English letter (different asterisks might be replaced with different letters).

Katie has a favorite string s and a not-so-favorite string t and she would love to recover the mysterious code so that it has as many occurrences of s as possible and as little occurrences of t as possible. Formally, let's denote $f(x, y)$ as the number of occurrences of y in x (for example, $f(aababa, ab) = 2$). Katie wants to recover the code c' conforming to the original c , such that $f(c', s) - f(c', t)$ is largest possible. However, Katie is not very good at recovering codes in general, so she would like you to help her out.

Input

The first line contains string c ($1 \leq |c| \leq 1000$) — the mysterious code. It is guaranteed that c consists of lowercase English characters and asterisks "*" only.

The second and third line contain strings s and t respectively ($1 \leq |s|, |t| \leq 50$, $s \neq t$). It is guaranteed that s and t consist of lowercase English characters only.

Output

Print a single integer — the largest possible value of $f(c', s) - f(c', t)$ of the recovered code.

input
***** katie shiro
output
1

input
caat caat a
output
-1

input
a bba b
output
0

input
*** cc z
output
2

In the first example, for c' equal to "katie" $f(c', s) = 1$ and $f(c', t) = 0$, which makes $f(c', s) - f(c', t) = 1$ which is the largest possible.

In the second example, the only c' conforming to the given c is "caat". The corresponding $f(c', s) - f(c', t) = 1 - 2 = -1$.

In the third example, there are multiple ways to recover the code such that $f(c', s) - f(c', t)$ is largest possible, for example "aaa", "aac", or even "zaz". The value of $f(c', s) - f(c', t) = 0$ for all of these recovered codes.

In the fourth example, the optimal recovered code c' would be "ccc". The corresponding $f(c', s) - f(c', t) = 2$.